Embedded System Lab Manual

LAB EXPERIMENT NO. 7

Serial UART DMA STM32 Programming with STM32F103C8T6 Blue Pill C++

Submitted To:

Sir Yasir

Submitter by:

Ahsan Basharat

Date of Submission:

7/06/2026

Introduction

In this lab, we learned how to use UART communication with DMA (Direct Memory Access) on the STM32 microcontroller. We studied how DMA transfers data between memory and the UART peripheral without continuous CPU involvement, making communication faster and more efficient. We configured UART and DMA using STM32CubeIDE and the HAL library, implemented data transmission and reception, and observed how DMA reduces CPU workload while improving overall system performance.

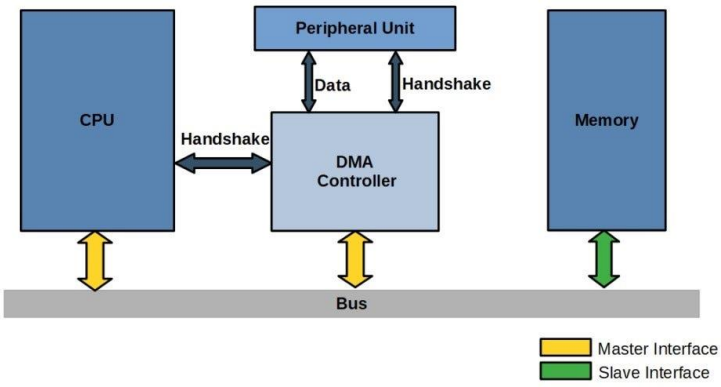
Direct Memory Access (DMA)

DMA is a bus controller and system peripherals providing high speed data transfer between peripherals and memory, as well as memory to memory.

- DMA transfer controlled by DMA Function / Controller, which request control of the bus from CPU.
- Data can quickly and easily transfer without CPU action.

Working of DMA

While the DMA working on system bus the CPU do other tasks where system bus is not needed. First When the peripheral need input or output they request DMA and DMA request to CPU to HOLD the system bus, CPU send two things to DMA one is i/o source address and Data count and then allow permission to HALD the system bus and then Peripherals can be share data through DMA on this time CPU complete those works where they don't need system bus.



On the simple way DMA and CPU perform TWO WAY handshake where DMA need ACK and CPU allow ACK.

When the DMA transfer is complete, it releases the system bus so the CPU can continue its execution. DMA takes control of the bus only when it needs to transfer data and releases it after the transfer is finished (or between transfers, depending on the DMA mode).

Mode of DMA Transfer

There are three modes of DMA transfer,

Burst Mode

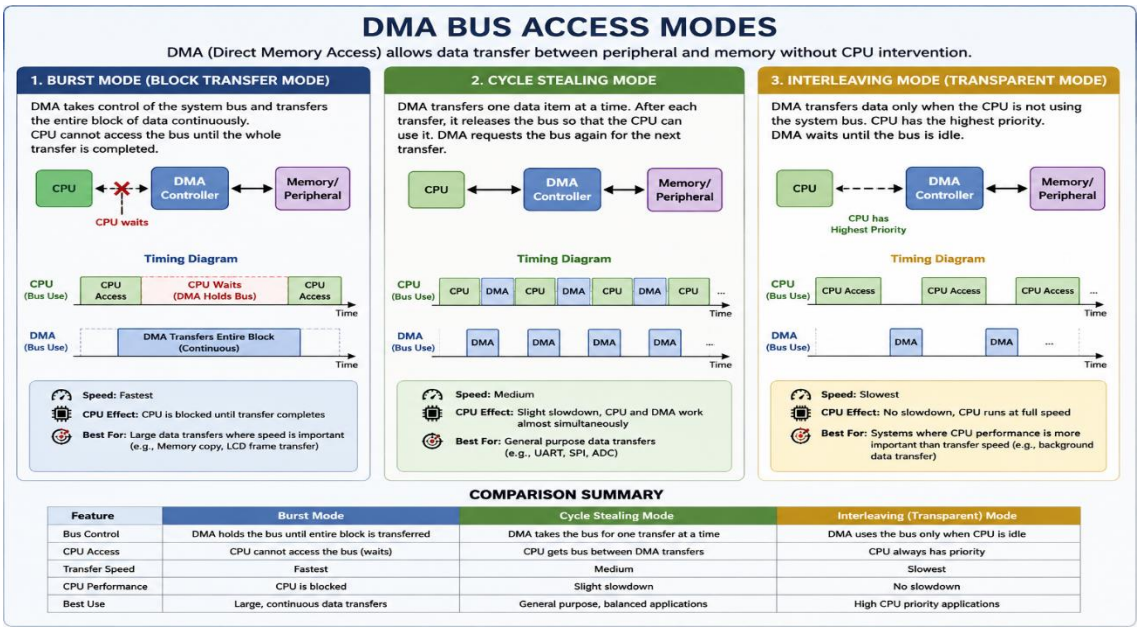
In Burst Mode, the DMA controller gains control of the system bus and transfers an entire block of data continuously before releasing the bus. During this time, the CPU cannot access the system bus and must wait until the DMA finishes the transfer.

Cycle Stealing Mode

In Cycle Stealing Mode, the DMA transfers only one data item (such as one byte, half-word, or word) at a time. After each transfer, it immediately releases the system bus so that the CPU can continue its operations. Whenever another data transfer is required, the DMA requests the bus again. This allows both the CPU and DMA to work almost simultaneously, with only a slight reduction in CPU performance

Interleaving Mode

In Interleaving Mode the DMA transfers data only when the CPU is not using the system bus. The CPU always has higher priority, so it continues executing without any interruption. The DMA waits until the bus is free and then transfers the data. Because the DMA can transfer data only during the CPU's idle time, the transfer is slower. This mode is used when keeping the CPU running at full performance is more important than transferring data quickly.



HAL_DMA_Functions

HAL_DMA_Init()

This function is used to Initializes the DMA channel.

HAL_StatusTypeDef HAL_DMA_Init(DMA_HandleTypeDef *hdma);

&hdma_usart1_tx

- A pointer to the DMA handle for USART1 Transmit (TX).
- This handle contains all DMA configuration parameters,

```
hdma_usart1_tx.Instance = DMA1_Channel4;
hdma_usart1_tx.Init.Direction = DMA_MEMORY_TO_PERIPH;
hdma_usart1_tx.Init.PeriphInc = DMA_PINC_DISABLE;
hdma_usart1_tx.Init.MemInc = DMA_MINC_ENABLE;
hdma_usart1_tx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
hdma_usart1_tx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
hdma_usart1_tx.Init.Mode = DMA_NORMAL;
hdma_usart1_tx.Init.Priority = DMA_PRIORITY_LOW;
```

HAL_DMA_Init() checks whether the DMA handle and its configuration are valid, then initializes the DMA hardware registers.

HAL_DMA_Start()

This function starts the DMA transfer by setting the source address, destination address, data length, and enabling the DMA channel.

HAL_DMA_Start(DMA_HandleTypeDef *hdma, uint32_t SrcAddress,uint32_t DstAddress,uint32_t DataLength);

- hdma – Pointer to the DMA handle.
- SrcAddress – Source address (where the data comes from).
- DstAddress – Destination address (where the data goes).
- DataLength – Number of data items to transfer.

HAL_UART_Transmit_DMA()

This function Sends data from memory to the UART peripheral using DMA.

UART_HandleTypeDef *huart,uint8_t *pData, uint16_t Size);

- huart → Pointer to the UART handle (e.g., &huart1).

- pData → Pointer to the data buffer to transmit.
- Size → Number of bytes to send.

For Example:

```
uint8_t txBuffer[] = "Hello";
HAL_UART_Transmit_DMA(&huart1, txBuffer, sizeof(txBuffer));
```

HAL_UART_Receive_DMA()

This function Receives a fixed number of bytes from UART into memory using DMA.

```
HAL_StatusTypeDef HAL_UART_Receive_DMA(UART_HandleTypeDef *huart,uint8_t *pData,uint16_t Size);
```

For Example

```
uint8_t rxBuffer[20];=
HAL_UART_Receive_DMA(&huart1, rxBuffer, 20);
```

HAL_UARTEx_ReceiveToIdle_DMA()

This Function Receives variable-length data using DMA and stops when the UART line becomes idle.

```
HAL_StatusTypeDef HAL_UARTEx_ReceiveToIdle_DMA(UART_HandleTypeDef *huart,uint8_t *pData,uint16_t Size);
```

For Example

```
uint8_t rxBuffer[100];
HAL_UARTEx_ReceiveToIdle_DMA(&huart1, rxBuffer, 100);
```